◇ Lightning Labs

Agentic Formal Methods for the Lightning Network

Prose specs hide ambiguity. A P model of channel splicing — built, checked, and bridged by an agent — exposes it.

Olaoluwa Osuntokun — Lightning Labs

Prose Specs Hide Ambiguity. Agents Expose It.

Thesis. The BOLTs are English MUST/SHOULD/MAY prose — the worst medium for a consensus-critical, adversarial, asynchronous protocol. Every "SHOULD" is an interop fork; every unstated race is a funds-loss vector.

- The spec text is normative; the state machine it implies is *latent*.
- Translating that latent machine into an *executable* model forces every implicit assumption into the open.
- An LLM agent can do this across the *whole* corpus: model, run a checker, triage counterexamples, round-trip each to a `file:line` citation — coverage a human spec team rarely sustains.

The artifact already exists

A real P model of channel splicing lives in `models/`: **8 build phases**, **17 P tests + 3 Go bridge tests green**, **20 catalogued questions**, **10 findings** against the live spec.

Why Splicing Is the Hard Case

Lightning is money software with no rollback. Splicing is the subtlest sub-protocol in the BOLTs: five independent state machines interleave on the *live* funding output.

- **Quiescence** (`stfu`) — `02-peer-protocol.md:1490`. Pause the channel before any structural change.
- **Interactive-tx** — `02-peer-protocol.md:100`. Turn-based, RBF-able tx construction; shared with v2 dual-funding.
- **Splice glue** — `02-peer-protocol.md:1558`. Drives the above; tracks multiple in-flight commitments.
- **Reconnect** (`channel_reestablish`) — `02-peer-protocol.md:3360`. Retransmit half-signed splices.
- **Blockchain** — confirmation, RBF fee bumps, reorg, zeroconf.

The ambiguity lives in the cartesian product: quiescence × interactive-tx × RBF × reconnect × reorg. No worked example in `splicing-test.md` covers the whole space — a checker does.

What Is P?

P is a state-machine language for asynchronous, event-driven systems — from Microsoft Research, used heavily inside AWS to verify production systems (S3, EBS, and other storage/coordination services).

- You write **machines**: actors that receive events, mutate local state, and `send`/`announce` events — exactly the Lightning peer model.
- You write **spec monitors**: separate observers that watch the event stream and `assert` global invariants.
- The **checker** explores message interleavings exhaustively within bounds; a violated assertion yields a concrete, replayable counterexample.

Why it fits Lightning

Async message passing, adversarial scheduling (`Network` drops/reorders/disconnects), non-deterministic environment (`Blockchain` confirms/forks/reorgs) — all first-class in P.

Why “Agentic”

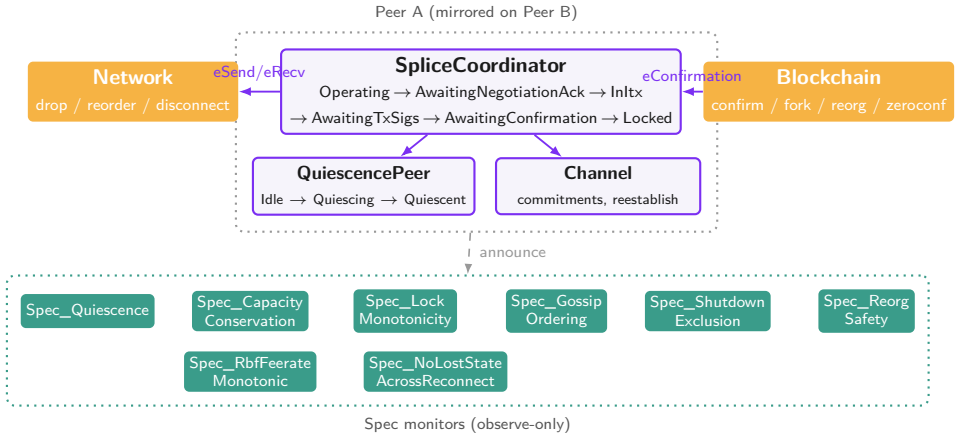
A human formalist models the section they're focused on. An agent models the *corpus* and keeps the model honest across phases:

1. **Reads the whole spec.** Surveys every relevant clause into `SPEC_SURVEY.md` with `file:line` citations first.
2. **Seeds hypotheses.** Pre-registers 18 candidate ambiguities in `SPEC_QUESTIONS.md` (Q1–Q18) — each falsifiable.
3. **Builds iteratively.** 8 phases; each adds machines/monitors and never breaks a prior phase's checks.
4. **Runs the checker, triages.** Maps each counterexample back to a clause; promotes real ambiguities to `FINDINGS.md`.
5. **Closes the loop.** Generates conformance traces from the model so a spec change regenerates the test vectors.

The hard part for humans is the bookkeeping — the agent owns the citation web (monitor, finding, question all cite a line) as the source of truth.

Model Architecture

One machine per sub-protocol per peer; `Network` and `Blockchain` are global; whole-system invariants live in dedicated monitors — not buried in machine code (`ARCHITECTURE.md`).



`splicing.pproj` — protocol machines + monitors. `infra.pproj` — `Network` + `Blockchain`, checkable in isolation.

Real P — The Quiescence State Machine

From `models/src/quiescence.p`. Each handler cites the BOLT clause it enforces; the assertion *is* the spec rule, executable.

```
machine QuiescencePeer {
  state Idle {
    on eOpenStfu do {
      // sec1508-1509: MUST NOT send stfu while HTLC/fee updates pending.
      if (hasPendingUpdates) { return; }
      assert !sentStfu, "sec1510: stfu sent twice";   // MUST NOT twice
      sentStfu = true; iSentInitiator = 1;
      send peerRef, eRecvStfu, (channel_id = 0, initiator = 1);
      goto Quiescing;
    }
  }
  state Quiescing {
    on eUpdateAttempted do {  // sec1517: MUST NOT send update after stfu
      assert false, "sec1517: update sent after our stfu";
    }
    on eDisconnect do {       // sec1529-1530: disconnect clears quiescence
      announce eQuiescenceCleared, (peer = pid,); goto Reset;
    }
  }
}
```

Real P — A Spec Monitor

Monitors are observe-only: they never drive the protocol, they watch the announced event stream and assert global invariants. From `models/src/monitors.p`:

```
// Spec_LockMonotonicity: once a peer emits splice_locked for a txid, it
// MUST NOT emit a commit_sig for a strict ANCESTOR of that splice.
// BOLT 2 sec2024-2028 ("stop commitment_signed for RBF attempts and
// ancestors once splice_locked is mutual").
spec Spec_LockMonotonicity
observes eSpliceLockedComplete, eCommitSigEmitted
{
  var locked: map[tPeerId, tTxid];
  start state Watching {
    on eSpliceLockedComplete do (e: (peer: tPeerId, txid: tTxid)) {
      locked[e.peer] = e.txid;
    }
    on eCommitSigEmitted do (e: (peer: tPeerId, funding_txid: tTxid)) {
      if (e.peer in locked) {
        assert e.funding_txid >= locked[e.peer],
          "commit_sig emitted for ancestor of locked splice";
      }
    }
  }
}
```

Spec Text → Model Assertion

The receiver-side `splice_init` rule, `02-peer-protocol.md:1666-1707`:

```
The receiving node:
  - If the channel is not quiescent: MUST warning/fail.       (sec1687)
  - If the sender is not the quiescence initiator: MUST fail. (sec1690)
  - If funding_contribution_satoshis is negative and |x| >
    the sender's current channel balance: MUST warning/fail.  (sec1704)
```

becomes, in `models/src/splice_coordinator.p`:

```
fun handlePeerNegotiationInit(contrib: tContributionSats, feerate: int) {
  peerContribution = contrib;
  if (contrib < 0 && (0 - contrib) > peerBalance) {
    // sec1704: |negative contribution| MUST NOT exceed sender balance.
    assert false, "sec1704: peer contribution exceeds peer balance";
  }
  // ... respond with splice_ack, goto InItxNonInitiator
}
```

§1687 / §1690 ("not quiescent" / "not initiator") are enforced *structurally*: `eRecvSpliceInit` is only handled after the local quiescence event lands — exactly Finding F1.

F1 — Quiescence/Coordinator Race

F1 **Open — spec gap (§1687, Q17a)**

Clause. `02-peer-protocol.md:1687` — receiver of `splice_init`: "If the channel is not quiescent: MUST . . . fail."

Ambiguity. Real implementations decouple the quiescence tracker from the splice coordinator. The local "we are now quiescent" notification and the peer's inbound `splice_init` arrive on different paths and *race*.

Model showed. The coordinator can process the wire `splice_init` *before* registering its own quiescence transition, then either:

- wrongly fire the §1687 branch and tear the channel down, or
- trust the wire and proceed — meaning **§1687 is unenforceable as written**.

Fix. Note at §1687 acknowledging the race; recommend either a unified peer-state lock, or treating the peer's wire as source of truth.

The model resolves it with `defer eRecvSpliceInit` in `Operating` — a normative MUST that is not enforceable against a conformant-but-differently-threaded peer.

F2 — `splice_locked` Arrives Mid-RBF Setup

F2 **Open (§1913, §527–528, §2032, Q18)**

Ambiguity. Peer A starts `tx_init_rbf`. Peer B's chain view has *already* hit acceptable depth on the original splice, so B sends `splice_locked` for the original exactly as A is mid-RBF setup. The spec scolds A for "RBF after splice_locked" — but A never sent `splice_locked`. **The spec is silent on what A does.**

Three defensible behaviors:

- Buffer + abandon the RBF (per §527–528 abandon-on-confirm).
- Continue the RBF; lock applies later.
- Treat as mismatched `splice_locked` (§2032) — but the txid is a legitimate splice, not a rival RBF candidate.

Model showed. It defers `eRecvSpliceLocked` in all pre-confirmation RBF states (`BufferedSpliceLocked`). Defensible — but unblessed.

Fix. A §1920-ish clause: "receiving `splice_locked` for a prior splice mid-`tx_init_rbf` ⇒ RBF MUST be abandoned; prior splice wins." A who-wins race — a correctness, not just liveness, gap.

F6 — Reserve Bypass on Pure Splice-Out

F6 **Open (§1818–1820 vs §1704–1707, Q10)**

The `tx_complete` reserve check fires *only*:

> `"Either side has added an output OTHER THAN the channel funding output and the balance for that side is less than the channel reserve ..."` (sec1818)

A *pure* splice-out takes funds via the change in the funding output itself — no extra output — so it **bypasses the reserve check**. The `splice_init`-receive check (§1704) only enforces $|\text{negative contribution}| \leq \text{balance}$, never reserve.

Model showed. Reproducer `tcSpliceOutHappy` with a contribution equal to the entire balance — **the model accepts it**. You can splice yourself below (or to zero) channel reserve on the new commitment.

Fix. Tighten §1818 to apply whenever the resulting balance dips below reserve regardless of extra outputs; or extend the §1707 receive-side check.

F7 — “Acceptable Depth” Is Undefined

F7 Open — one-line fix (§2014, Q5)

```
Each node:
  - If any splice transaction reaches acceptable depth:
    - MUST send ‘splice_locked‘ with the ‘txid‘ of that tx.   (sec2014)
```

Ambiguity. “Acceptable depth” is never defined. Is it the channel’s negotiated `minimum_depth` (as `channel_ready` uses)? Or separately negotiable for splices? The splice section never says.

Model showed. Uses `depth >= 6` as a placeholder. Two implementations with different thresholds open a window where A is sending `splice_locked` and B has not yet acknowledged — a benign-looking interop stall that pairs with the F2 / F4 mismatch races.

Fix. Add to §2014: “Acceptable depth equals the channel’s negotiated `minimum_depth`.”

The precondition shared by gossip ordering, reconnect’s `my_current_funding_locked`, and the lock-mismatch findings. Undefined here means undefined everywhere downstream.

F8 — Zeroconf Splice Double-Spend

F8 **Open — candidate funds-loss (§2016–2017, Q17)**

```
- If ‘option_zeroconf‘ has been negotiated:
  - SHOULD send ‘splice_locked‘ immediately after
    exchanging ‘tx_signatures‘.                    (sec2016)
```

and `tx_init_rbf` is *forbidden* under `option_zeroconf` (§1914, §1956–1959).

Ambiguity. What if the zeroconf splice tx is **double-spent on chain** by a malicious peer's pre-signed alternative? RBF — the natural fee-bump recovery — is unavailable, and the spec documents no fallback.

Model showed. This is the one finding the model does *not* yet exercise — honesty matters. Catalogued (Q17), flagged as the highest-value next scenario *because* it is a candidate funds-loss vector.

Fix. Either recommend against `option_zeroconf` for splices, or document recovery (unilateral close from the prior funding tx).

F9/F10 — The Foundation Isn't Specified Either

F9 Convergence asserted, not specified (§2553–2571) **F10** Reconnect crossing (§3489–3503)

The splice model sits *on top of* the base commitment machine and abstracts it. The exercise surfaced that the base layer itself is under-specified — a deeper finding than any single splice clause.

The full-duplex lifecycle (§2558–2566). An update lands on the *other* node's commitment first; it is "irrevocably committed" only after a full `commit_sig`/`revoke_and_ack` round-trip in **both** directions — the predicate that gates HTLC forwarding.

> "As the two nodes' updates are independent, the two commitment transactions may be out of sync indefinitely. This is not concerning: what matters is whether both sides have irrevocably committed ..." (sec2568)

The spec asserts convergence with no checkable invariant. No normative claim — and no test vector — that *concurrent* `commit_sig` from both sides converges to one irrevocably-committed set (F9), nor that the reconnect counter crossing re-converges without losing an update (F10). **This is the next model.**

Deep Dive — The State Machine Beneath Splicing

F9/F10 were findings *about an abstraction boundary*. So we built the layer underneath: `channel.pproj` — the base full-duplex `commitment_signed` / `revoke_and_ack` protocol (BOLT 2 §2553–§2571).

- The splice model treated `commit_sig` / `revoke_and_ack` as one paired exchange. The real protocol is **two independent directions** — either peer can have a `commitment_signed` in flight at any time.
- The next slides show the actual P machine, the invariant that pins down the convergence the spec only asserts, and a concrete **fund-loss counterexample** the model produces on demand.

The meta-point: modeling the new feature (splicing) forced us to confront that it rests on a base-layer property the spec never makes checkable — so we went down a layer and checked it.

The Per-Update Lifecycle (§2558–§2566)

An HTLC update is not "added" atomically. It walks a five-stage lifecycle, landing on the *remote* commitment first and the *local* one only after a `revoke_and_ack`:

```
1. update_add_htlc    sent  ->  enters peer's "received" set
2. commitment_signed  sent  ->  peer adopts it (its local commitment)
3. revoke_and_ack     recv  ->  now on peer's commitment
4. commitment_signed  recv  ->  we adopt it (our local commitment)
5. revoke_and_ack     sent  ->  now on our commitment  (both: irrevocable)
```

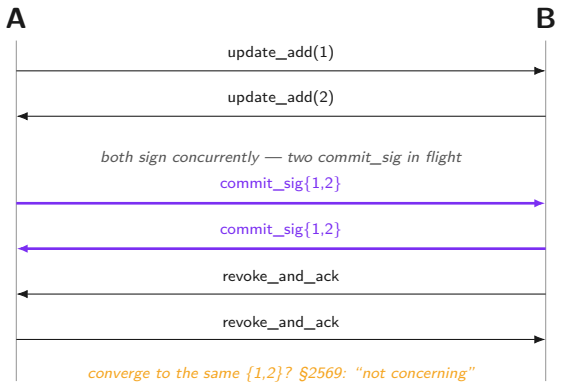
Only after stage 5 is the update **irrevocably committed** (§2570) — the single predicate that matters for safety. The asymmetry is the point: an update is real on one side before the other.

Modeling One Peer — `ChannelPeer`

```
machine ChannelPeer {
  var ownProposed:    set[tUpdateId]; // I sent update_add; pending on peer
  var theirReceived:  set[tUpdateId]; // peer's update_add I received
  var remoteSigned:   set[tUpdateId]; // I signed into peer's commitment
  var localCommitted: set[tUpdateId]; // peer signed into MY commitment
  var peerAcked:      set[tUpdateId]; // peer revoked-old: on peer's commitment
  ...
}
```

`localCommitted` ∩ `peerAcked` is the irrevocably-committed set in this peer's view (§2570). Each set advances on exactly one wire event — the lifecycle of the previous slide, made explicit. Compare to the splice model's two booleans: that compression is exactly what F9 says hides the convergence question.

Concurrent `commit_sig` — The Hard Case (§2568)



§2569 calls the out-of-sync condition "not concerning" — an *assertion*, not a theorem.

`tcConcurrentCommitSig` makes the checker prove convergence across every interleaving.

The Liveness the Spec Omits

The convergence check *failed first* — and the counterexample was instructive. A node that sends `commitment_signed` once, when it has no unsigned changes (empty cover), and never retries, strands the peer's update forever.

```
fun flushCommit() {
  // sec3106: MUST NOT send commit_sig with no updates (empty cover => no-op).
  // Idempotent: once everything known is signed this is a no-op => terminates.
  ... compute cover of un-signed updates ...
  if (sizeof(cover) == 0) { return; }
  send peerRef, eRecvCommitSig, (covered = cover,);
}
```

Convergence holds **iff** a node with pending changes *eventually* sends `commitment_signed`. BOLT 2 says when you MAY / MUST NOT (§3106) — never that you MUST eventually. **The omitted liveness obligation is the finding.**

The Convergence Invariant, Made Checkable

```
spec Spec_CommitmentConvergence observes eUpdateProposed, eIrrevocable {
  start cold state Converged { ... }
  hot   state Diverging  { ... }   // MUST eventually leave this state
  fun decide() {
    if (covers(irrevA, proposed) && covers(irrevB, proposed)) {
      goto Converged;              // both peers hold every proposed update
    } else { goto Diverging; }
  }
}
```

A P **liveness** monitor: `Diverging` is `hot` — a run that stays there forever is a bug. This is the precise, machine-checkable form of §2569's prose. Result: **green at 3000 schedules**, all interleavings.

The Irrevocably-Committed Predicate (§2570)

```
fun checkIrrevocable() {
  foreach (u in localCommitted) {
    if ((u in peerAcked) && !(u in announcedIrrevocable)) {
      announcedIrrevocable += (u);
      announce eIrrevocable, (peer = pid, id = u);  // safe to act on now
    }
  }
}
```

An update is irrevocable **only** when it is on my commitment (`localCommitted`) *and* the peer has revoked-old for it (`peerAcked`). One line — `(u in localCommitted) && (u in peerAcked)` — is the load-bearing safety predicate of the entire protocol. The next slide is what happens when a node skips it.

The Fund-Loss Counterexample — `tcForwardTooEarly`

A routing node that forwards an incoming HTLC before its *outgoing* update is irrevocable has **paid downstream but cannot claim upstream** (§3173–§3176). We model the unsafe node explicitly:

```
on eUserForward do (e: (id: tUpdateId, eager: bool)) {
  if (e.eager) {                          // UNSAFE node: forward NOW
    announce eForwardAttempt,
      (peer = pid, id = e.id,
       isIrrevocable = (e.id in localCommitted) && (e.id in peerAcked));
  } else { pendingForwards += (e.id); tryForward(); }  // CONFORMANT: wait
}
```

`Spec_ForwardOnlyIrrevocable` asserts every forward is irrevocable. `tcForwardTooEarly` (`eager=true`) **must** find the violation — a checked-in negative test, the fund-loss demonstration on demand.

Conformant vs. Unsafe, Side by Side

Test	Node behavior	Monitor result
`tcForwardSafe`	holds forward until irrevocable	green (no violation)
`tcForwardTooEarly`	forwards eagerly (unsafe)	**bug** (must fire)

The model encodes *both* a conformant and a non-conformant node, and the suite asserts the monitor distinguishes them. A spec that only describes the conformant path can't tell you the cost of the other one; the model makes the cost a **failing test**.

The negative test proves the monitor has teeth — that `Spec_ForwardOnlyIrrevocable` would actually catch a real implementation that got this wrong.

F10 — Carrying Convergence Across a Reconnect

F9 is steady-state convergence. F10 is the *same property across a disconnect* — historically the most bug-prone region (§3489–§3554):

- `next_commitment_number` / `next_revocation_number` crossing decides who retransmits what.
- Retransmit `revoke_and_ack` and `commitment_signed` "in the same relative order as initially transmitted" (§3495–§3496).
- The asymmetric `next_revocation_number` ± 1 reasoning (§3498–§3503).

Modeled. `tcReconnectMidCommit` disconnects both peers mid-commit (un-acked `commit_sig` dropped), reconnects, and `Spec_CommitmentConvergence` still discharges — across $\sim$**1128 timelines**. This checks the *post-condition* the spec states only imperatively; the literal counter arithmetic is the next refinement.

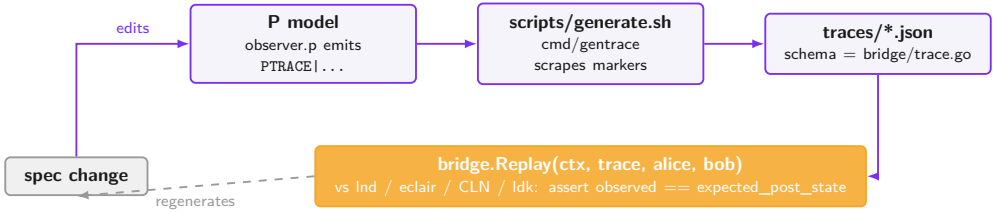
What the Channel Model Buys Us

- **Convergence is now machine-checked**, not asserted — across all concurrent-`commit_sig` interleavings (§2569 made into a theorem).
- **The omitted liveness obligation is named** — a node MUST eventually flush pending changes, or convergence fails (F9).
- **A fund-loss vector is a one-command reproducer** — `tcForwardTooEarly` (§3173–§3176).
- **The abstraction boundary itself became a finding** — modeling splicing forced the base layer into scope.

A good model doesn't stop at the feature you set out to verify — it tells you which of your foundations were only ever assumed.

The Bridge — Closing the Loop

The model is only useful if real clients can be tested against it. Phase 7 ships a Go replay harness (`models/bridge/`). Traces are **auto-generated from the model**, not hand-authored.



- Traces **auto-generated from the model** — a spec/model change *regenerates* them; `go test ./bridge` catches drift.
- Each `Event` carries its `spec_citation` (`file:line`); a replay mismatch points straight at the BOLT clause.
- The `Implementation` interface is the only thing a client implements; a `MockImplementation` passes every canonical trace today.

Results

Phase	Spec area	P tests	Max sched.	Result
1	Quiescence (`stfu`)	5	2000	green
2	Happy-path single splice	3	2000	green
3	RBF + multi-pending commitments	3	2000	green
4	Disconnect / `channel_reestablish`	1	3000	green
5	Blockchain non-determinism (reorg)	1	2000	green
6	Channel close + gossip post-splice	1	2000	green
7	Bridge (Go replay harness)	3 (Go)	n/a	green
9	Base commitment FSM (F9/F10)	5 + 1*	3000	green

- **21 P tests + 3 Go bridge tests, all green** (* + one checked-in *counterexample*, `tcForwardTooEarly`, which MUST fire).
- ∼**1128 timelines** across a reconnect (`tcReconnectMidCommit`); ∼88 in the deepest splice scenario.
- **20 catalogued questions** (Q1–Q20) and **10 findings** (F1–F10) — F1–F8 splice-clause ambiguities, F9/F10 the foundational commitment-FSM gap (convergence now machine-checked).

Limitations (Stated Honestly)

The model is a **companion to the spec, not a replacement**. It deliberately abstracts:

- **Cryptography.** Signatures, nonces, key material are boolean validity flags (`has_shared_input_sig`), not real secp256k1 / MuSig2. Taproot nonce coordination (`bolt-simple-taproot.md:1135-1172`) is surveyed, not yet checked.
- **Exact wire encoding.** Txids/amounts are small symbolic integers (`tTxid = int`); the model proves *protocol logic*, not byte-level serialization.
- **Underspecified constants as config flags.** "Acceptable depth" → `depth >= 6`; v2 tied-funder, RBF re-quiescence, and mismatch buffer-vs-drop are `tModelConfig` modes.
- **The base commitment FSM.** `commit_sig` / `revoke_and_ack` is a paired exchange, not the real per-update lifecycle. That abstraction is what F9/F10 name — and the next model.

F1–F8 don't depend on the abstracted layers — pure protocol-logic gaps. F9/F10 are about the abstracted base layer itself, which is precisely why modeling it is the next step.

Vision — The Spec Model as a CI Gate

Make the loop standard practice:

1. **Every spec PR ships a model delta.** The decomposition mirrors BOLT sections, so a PR touching one section maps to one model file — minimal blast radius.
2. **The checker gates ambiguity.** If a clause admits two readings, the model carries both as a `tModelConfig` mode and the ideal monitor must hold under each. A violating reading is a counterexample, filed before merge.
3. **Regenerate vectors automatically.** `scripts/generate.sh` re-emits the JSON traces; `go test ./bridge` catches drift.
4. **Cross-implementation conformance becomes continuous.** lnd / eclair / CLN / ldk replay the regenerated vectors via one interface; a divergence reports the exact `file:line` it broke.

The spec stops being prose-with-test-vectors-bolted-on and becomes **prose + executable model + auto-derived conformance suite** — all citation-linked, all in one repo, all gated in CI.

Call to Action

- **Read the artifact.** `models/FINDINGS.md` (F1–F10) and `models/SPEC_QUESTIONS.md` (Q1–Q20) are ready for review today.
- **Land the cheap fixes first.** F7 (define "acceptable depth" = `minimum_depth`) and F6 (reserve check on pure splice-out) are one-clause edits.
- **Adjudicate the races.** F1 (quiescence/coordinator), F2 (RBF vs `splice_locked`), F8 (zeroconf double-spend) need a normative ruling.
- **Model the foundation (F9/F10).** Build the base commitment-FSM model — concurrent `commit_sig` convergence + reconnect crossing — and state the convergence guarantee normatively.
- **Adopt the loop for the next sub-protocol.** Same pipeline at PTLCs / attributable failures — survey, model, check, bridge, gate.

The ask

Treat the model in `models/` as the reference companion to the splicing spec, and require a **model delta + regenerated vectors** on the next splicing-related BOLT PR.